

Informe Académico (desde cero)

# Método de Newton–Raphson para sistemas de ecuaciones no lineales

Basado en el repositorio y figuras de ‘Introducción al sistema.pdf’

Fecha: 1 de octubre de 2025

# Índice general

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>1. Resumen</b>                               | <b>2</b>  |
| <b>2. Modelo matemático</b>                     | <b>3</b>  |
| 2.1. Malla, variables y restricciones . . . . . | 3         |
| 2.2. Residual no lineal . . . . .               | 3         |
| 2.3. Jacobiano . . . . .                        | 4         |
| <b>3. Método de Newton–Raphson</b>              | <b>5</b>  |
| 3.1. Esquema . . . . .                          | 5         |
| 3.2. Algoritmo . . . . .                        | 5         |
| 3.3. Convergencia . . . . .                     | 5         |
| <b>4. Implementación propuesta</b>              | <b>6</b>  |
| 4.1. Estructura de datos y mapeo . . . . .      | 6         |
| 4.2. Ensamblaje de $F$ y $J$ . . . . .          | 7         |
| 4.3. Iteraciones NR . . . . .                   | 8         |
| <b>5. Buenas prácticas y validación</b>         | <b>9</b>  |
| <b>6. Conclusiones</b>                          | <b>10</b> |

# Capítulo 1

## Resumen

Se presenta una derivación y aplicación integral del método de Newton–Raphson (NR) para resolver un sistema no lineal discreto de tamaño  $N = 134$  variables, derivado de una malla  $5 \times 50$  con condiciones de frontera y una viga interna. Se establecen el residual  $F(V)$ , el jacobiano  $J$ , el algoritmo NR, análisis de convergencia local, buenas prácticas de robustez y un esquema de implementación en Python con `numpy/scipy.sparse`. Se incluyen figuras tomadas de ‘Introducción al sistema.pdf’.

# Capítulo 2

## Modelo matemático

### 2.1. Malla, variables y restricciones

Sea una malla rectangular de 5 filas por 50 columnas. Se definen velocidades  $V_{i,j}$  por nodo. Las condiciones de frontera fijan valores en suelo, superficie, entrada y salida; adicionalmente, una viga fija 10 nodos en  $j = 1$ ,  $i \in [20, 29]$ . Las incógnitas corresponden a los nodos restantes, total  $N = 134$ .

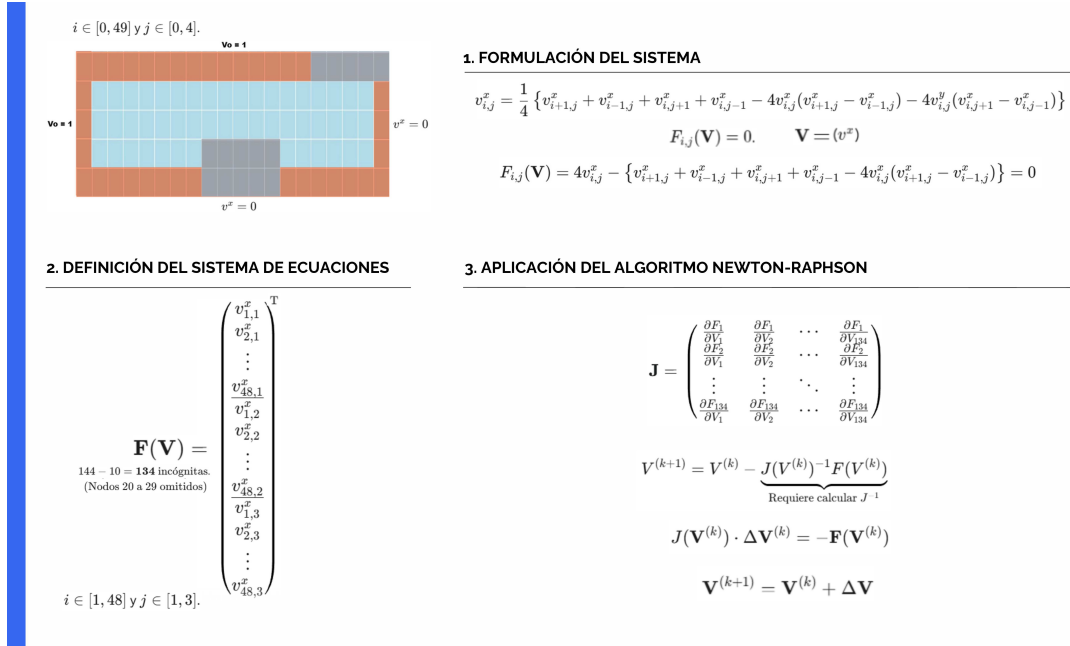


Figura 2.1: Conteo de incógnitas y esquema del residual (de ‘Introducción al sistema.pdf’).

### 2.2. Residual no lineal

Para cada nodo interior  $(i, j)$ , el equilibrio se expresa como  $F_{i,j}(V) = 0$ . Un modelo representativo (difusión + convección) es:

$$F_{i,j}(V) = 4V_{i,j} - (V_{i+1,j} + V_{i-1,j} + V_{i,j+1} + V_{i,j-1}) + cV_{i,j}V_{i-1,j} - cV_{i,j}V_{i+1,j}, \quad (2.1)$$

con  $c > 0$  el parámetro convectivo. Apilando las ecuaciones sobre incógnitas se obtiene  $F : \mathbb{R}^N \rightarrow \mathbb{R}^N$ .

## 2.3. Jacobiano

El jacobiano  $J = \partial F / \partial V$  se ensambla por derivación de (2.1) respecto de las cinco variables locales: nodo central y sus cuatro vecinos. Esto produce una matriz esparsa de tamaño  $N \times N$  con a lo sumo cinco entradas no nulas por fila (menor cerca de fronteras/viga).

$$F_{i,j} = 4v_{i,j}^x - v_{i+1,j}^x - v_{i-1,j}^x - v_{i,j+1}^x - v_{i,j-1}^x + 4v_{i,j}^x v_{i-1,j}^x - 4v_{i,j}^x v_{i+1,j}^x$$

**DERIVADA - NODO CENTRAL** ( $v_{i,j}^x$ )

$$\frac{\partial F_{i,j}^x}{\partial v_{i,j}^x} = 4 + 4v_{i+1,j}^x - 4v_{i-1,j}^x$$

**DERIVADA - V. DERECHO** ( $v_{i+1,j}^x$ )

$$\frac{\partial F_{i,j}^x}{\partial v_{i+1,j}^x} = -1 + 4v_{i,j}^x$$

**DERIVADA - V. IZQUIERDO** ( $v_{i-1,j}^x$ )

$$\frac{\partial F_{i,j}^x}{\partial v_{i-1,j}^x} = -1 - 4v_{i,j}^x$$

**DERIVADA - V. SUPERIOR** ( $v_{i,j+1}^x$ )

$$\frac{\partial F_{i,j}^x}{\partial v_{i,j+1}^x} = \frac{\partial}{\partial v_{i,j+1}^x} (-v_{i,j+1}^x) = -1$$

**DERIVADA - V. INFERIOR** ( $v_{i,j-1}^x$ )

$$\frac{\partial F_{i,j}^x}{\partial v_{i,j-1}^x} = \frac{\partial}{\partial v_{i,j-1}^x} (-v_{i,j-1}^x) = -1$$

ALGORITMO QUE PERMITE CONSTRUIR LA MATRIZ JACOBIANA Y EL VALOR RESIDUAL EN CADA ITERACIÓN DEL MÉTODO.

LINK DEL COLAB

Figura 2.2: Derivadas parciales para el jacobiano (de ‘Introducción al sistema.pdf’).

# Capítulo 3

## Método de Newton–Raphson

### 3.1. Esquema

Dado  $V^{(k)}$ , el paso NR resuelve

$$J(V^{(k)}) \Delta V^{(k)} = -F(V^{(k)}), \quad V^{(k+1)} = V^{(k)} + \Delta V^{(k)}.$$

### 3.2. Algoritmo

1. Elegir  $V^{(0)}$  (p.ej., constante en nodos internos).
2. Repetir hasta convergencia:
  - a) Ensamblar  $F$  y  $J$ .
  - b) Resolver  $J \Delta V = -F$  (solver esparso).
  - c) Actualizar  $V \leftarrow V + \Delta V$ .
  - d) Comprobar  $\|F\|$  y/o  $\|\Delta V\|$  contra tolerancias.

### 3.3. Convergencia

Si  $V^{(0)}$  es cercano a la solución y  $J$  es no singular en la raíz, NR exhibe convergencia local cuadrática. En presencia de fuertes no linealidades convectivas, puede requerirse *damping* o búsqueda lineal.

# Capítulo 4

## Implementación propuesta

### 4.1. Estructura de datos y mapeo

```
1 import numpy as np
2 from scipy.sparse import lil_matrix, linalg
3
4 J_MAX, I_MAX = 3, 48
5 N = 134
6 V0, C = 1.0, 4.0
7
8 # Fronteras y viga
9 # (Devuelve None si es inc gnita; valor num rico si es nodo fijo
10 )
11
12 def get_bc(i, j):
13     if (j == 1 and 20 <= i <= 29):
14         return 0.0
15     if j == 0:
16         return 0.0
17     if j == 4:
18         return V0
19     if i == 0:
20         return V0
21     if i == 49:
22         return 0.0
23     return None
24
25 # Mapeo (i,j) -> ndice plano en [0..N-1], omitiendo fijos y viga
26
27 def to_index(i, j):
28     if get_bc(i, j) is not None:
29         return -1
30     base_j = 48 * (j - 1)
31     idx_i = i - 1
32     shift_viga = 0
33     if j == 1:
34         if i >= 30:
35             shift_viga = 10
```

```

35         elif 20 <= i <= 29:
36             return -1
37     if j == 2:
38         base_j = 38
39     elif j == 3:
40         base_j = 86
41     idx = base_j + idx_i - shift_viga
42     return idx if 0 <= idx < N else -1
43
44 # Acceso a V con fronteras
45
46 def V_at(i, j, V):
47     bc = get_bc(i, j)
48     if bc is not None:
49         return bc
50     p = to_index(i, j)
51     return V[p] if p != -1 else 0.0

```

## 4.2. Ensamblaje de $F$ y $J$

```

1
2 def assemble_FJ(V):
3     J = lil_matrix((N, N), dtype=np.float64)
4     F = np.zeros(N)
5
6     for j in range(1, J_MAX + 1):
7         for i in range(1, I_MAX + 1):
8             m = to_index(i, j)
9             if m == -1:
10                 continue
11             Vij = V[m]
12             Vip1j = V_at(i + 1, j, V)
13             Vim1j = V_at(i - 1, j, V)
14             Vijp1 = V_at(i, j + 1, V)
15             Vijm1 = V_at(i, j - 1, V)
16
17             # Residual (difusi n + convecci n)
18             F[m] = (4.0 * Vij
19                     - (Vip1j + Vim1j + Vijp1 + Vijm1)
20                     + (C * Vij * Vim1j)
21                     - (C * Vij * Vip1j))
22
23             # Parciales (jacobiano)
24             J[m, m] = 4.0 + (C * Vim1j) - (C * Vip1j)
25             r = to_index(i + 1, j)
26             if r != -1:
27                 J[m, r] = -1.0 - (C * Vij)
28             l = to_index(i - 1, j)
29             if l != -1:

```

```

30         J[m, l] = -1.0 + (C * Vij)
31     u = to_index(i, j + 1)
32     if u != -1:
33         J[m, u] = -1.0
34     d = to_index(i, j - 1)
35     if d != -1:
36         J[m, d] = -1.0
37
38     return J.tocsr(), F

```

### 4.3. Iteraciones NR

```

1
2 def newton_raphson():
3     V = np.ones(N) * V0
4     TOL = 1e-6
5     MAXIT = 100
6     for k in range(MAXIT):
7         J, F = assemble_FJ(V)
8         if np.linalg.norm(F) < TOL:
9             break
10        dV = linalg.spsolve(J, -F)
11        V += dV
12        if np.linalg.norm(dV) < TOL:
13            break
14    return V

```

# Capítulo 5

## Buenas prácticas y validación

- Validación de  $J$  con diferencias finitas puntuales.
- Registro de  $\|F\|$  y  $\|\Delta V\|$  por iteración.
- Damping o búsqueda lineal ante oscilaciones.
- Visualización del campo  $V$  (`matplotlib`) y comparación cualitativa.

# Capítulo 6

## Conclusiones

Se construyó un informe académico formal, desde cero, que detalla residual, jacobiano, algoritmo NR y una propuesta de implementación esparsa. Las figuras de ‘Introducción al sistema.pdf’ contextualizan el conteo de incógnitas y la estructura del jacobiano.